Chapter 8

Support Vector Machines

8.1 Introduction to SVM

The field of '**statistical learning theory**' began with Vapnik and Chervonenkis (1974) [1] and SVMs can be said to have started when statistical learning theory was developed. SVMs close to their current form were first introduced with a paper in 1992 at the Conference on Computational Learning Theory [2]. Since then it is being highly used in various applications like handwritten recognition, face identification, Computational linguistics, Automatic Speech Recognition Systems. The general class of algorithms resulting from this process is known as kernel methods or kernel machines, though kernel methods in a different form were introduced way back in 1964 [3, 4].

The support vector machine (SVM) [5] is a relatively recent addition to the toolbox of the AI practitioner. Its development followed in the reverse order to the development of neural networks (NNs). SVMs evolved from the sound theory to the implementation and experiments, while the NNs followed more heuristic path, from applications and extensive experimentation to the theory. It is interesting to note that the very strong theoretical background of SVMs did not make them widely appreciated at the beginning. The publication of the first paper by Vapnik, Chervonenkis and co-workers[6,7] went largely unnoticed till 1992. This was due to a widespread belief in the statistical and/or machine learning community that, despite being theoretically appealing, SVMs are neither suitable nor relevant for practical applications. They were taken seriously only when excellent results on practical learning benchmarks were achieved (in numeral recognition, computer vision and text categorization). Today, SVMs show better results than (or comparable outcomes to) NNs and other statistical models, on the most popular benchmark problems.

8.1.1 Some Applications of SVM

- Handwriting recognition

- Natural Language Processing
- Computational Biology
- Computer Vision
- Text Mining
- Speech Recognition and Machine translation
- Drug Design
- Machine Fault Diagnostics

8.1.2 SVM Basics

The underlying concept of SVM algorithm lies within Linear Algebra and convex optimization. The machine learning problem is represented in a mathematical framework where each training data is taken as a feature vector and plotted in an 'n' dimensional space based on the feature values. Once all the training data is represented in the **input space**, the aim of the SVM algorithm is to classify the data points of different classes by finding the classifier function between them. Once the classifier is found out, the algorithm can do the prediction based on the classifier function. Finding out the classifier is the real task and it is performed by solving a convex optimization problem.

Please note that the below explanation is just an overview of the SVM knowledge required for understanding the problems done using excel, for more details you can refer the book [5].

Let's now understand this concept by using a simple example.

8.1.2.1 SVM – a simple example

Let us take two types of fruits. One is apple and other one is orange. Our problem is to make machine (Computer) learn these fruits so that it can predict it correctly in the future.

We divide the entire process into two phases **training** and **testing**. Before starting, there is a pre-processing step which is common to all machine learning problems and that is called feature extraction. It is one crucial step which decides the overall accuracy of the application.

Here we deduce some of the distinguishable attributes or features which separate one class of data from other. More the contrasting features we have, more the accuracy will be for our application. Good features will help in finding a good classifier function. For a machine learning algorithm, both learning and testing data is provided in terms of feature vectors.

Let's go back to the original example. Here we take two common features associated with these fruits like its color and no of seeds. We provide the system with some training examples along with its class names i.e. apple or orange.

Table 8.1.1 is the training data in tabular format.

Sno	Color	No of Seeds	Class
1	Orange	7	Orange
2	Red	3	Apple
3	Red	2	Apple
4	Orange	10	Orange
5	Red	4	Apple

Table 8.1.1 Input training data

Let's plot the data in 2D plane (Figure. 8.1.1). Assume Orange color corresponds to value 3 and Red for value 6.

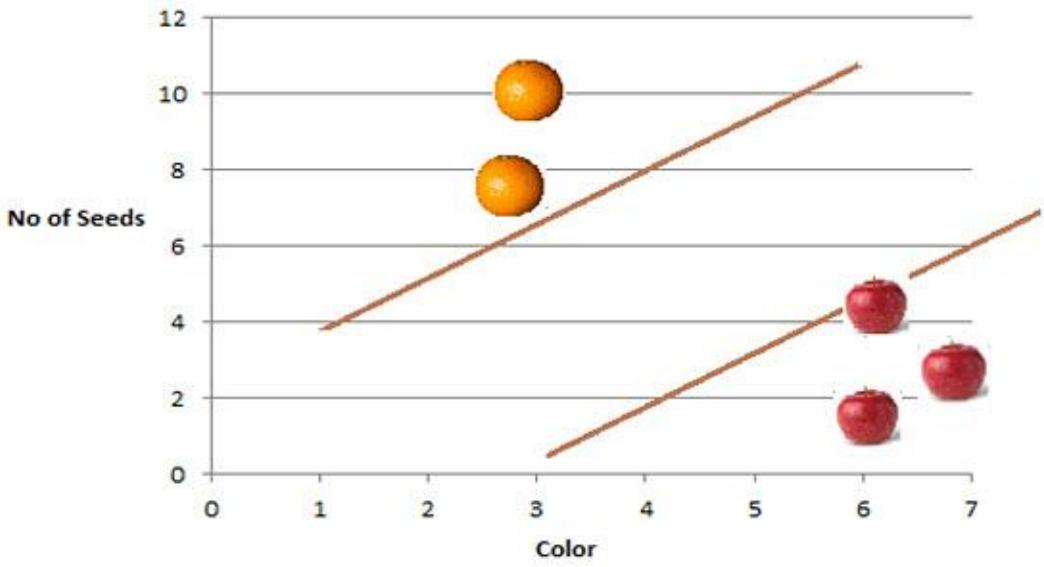


Fig. 8.1.1 2D plot of the data present in Table 8.1.1

Seeing the plot in figure 8.1.1, one can clearly separate oranges and apples by drawing two bounding planes (brown color). In practical applications, the bounding planes **(hyperplanes)** are achieved by solving a quadratic convex optimization problem (discussed later). Classifier function is taken as the line from the midway of bounding planes.

In **testing phase**, the classifier function obtained from the training stage is used for predicting the class of new input data. In the example discussed above, machine would be substituting the feature values of new input data into the classifier function. Based on the sign of the resultant value, the machine will predict whether the new test fruit is apple or an orange.

8.1.3 SVM formulation and Computation

Before heading our way towards formulating the actual problem, let us see the situations/cases of different machine learning problems.

Situation 1: The training data present in the input space is clearly separated among different classes (Figure. 8.1.2).

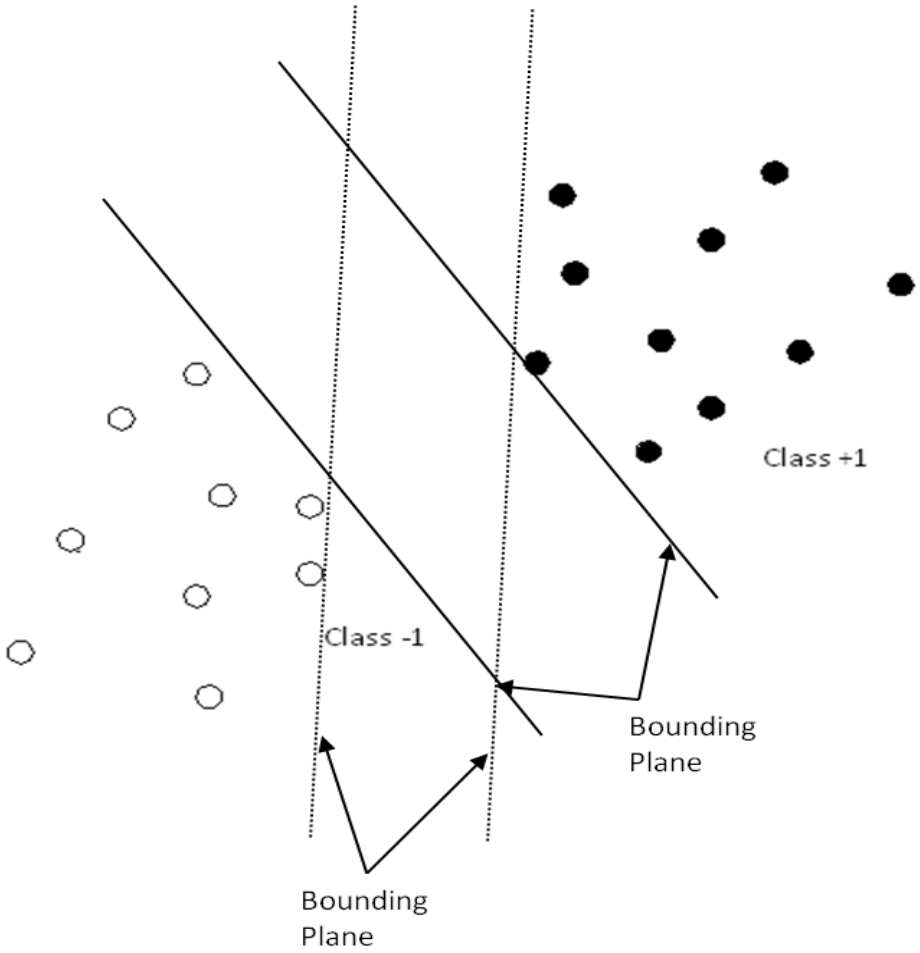


Fig. 8.1.2 Linear Separable Data

Here we see that it is very simple to identify bounding planes which can clearly separate (without any errors) both classes. But there are many pairs of bounding planes that does so as shown in figure 8.1.3. The question is which line should we use?

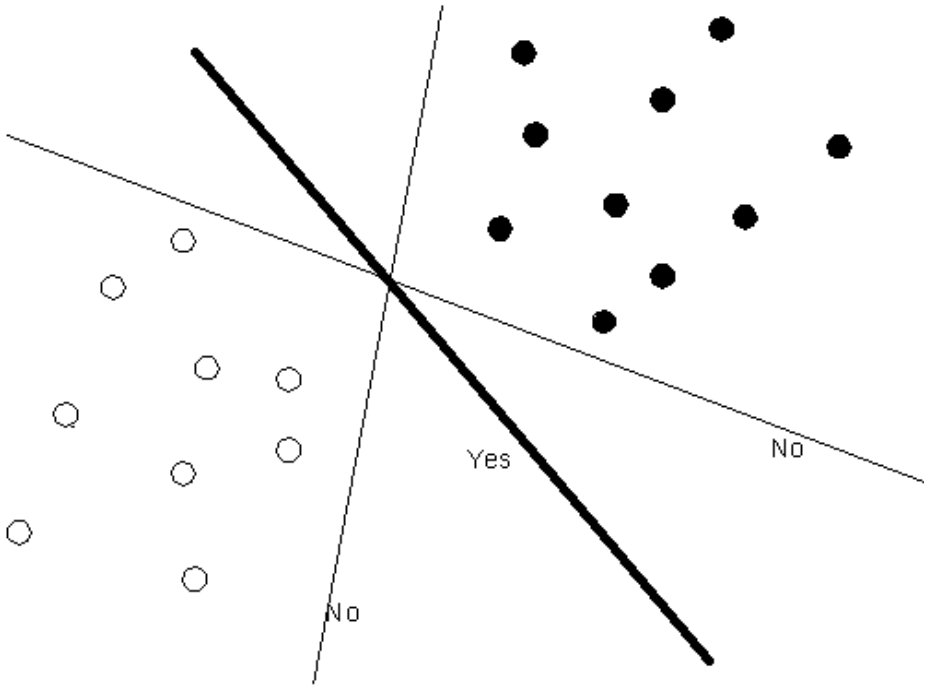


Fig. 8.1.3 Choosing the best plane for classification

The answer to the question is that one should aim to achieve maximum separation (margin) between the classes and reduce generalization error. By the term generalization error, we mean that when new examples (data points with unknown class values) arrive for classification, the chance of making error in our prediction (of the class which it belongs) based on the learned classifier (hyperplane) should be minimum. Hence in the previous figure 8.1.3, we choose the central line as the classifier since it is at larger distance from the data points.

Figure 8.1.4 illustrate the concept of 'maximum margin'. The two planes parallel to the classifier and which passes through one or more points in the data set is called '***bounding planes***'. The distance between these bounding planes is called margin and by SVM 'learning', we mean, finding a hyperplane which maximizes this margin.

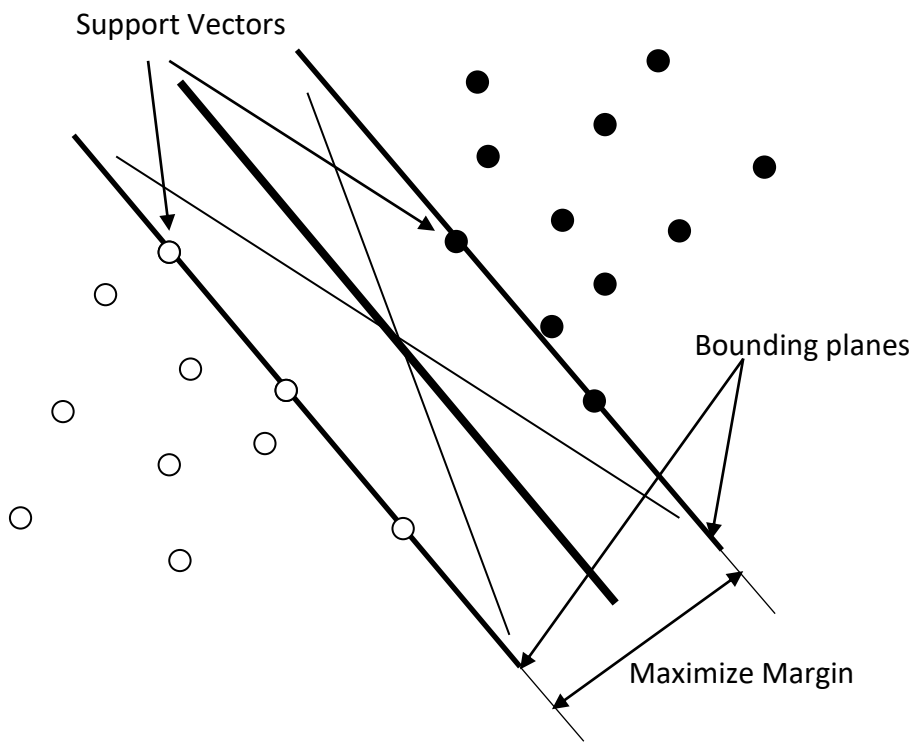


Fig. 8.1.4 A Maximal Margin Classifier (Printed with permission from [5])

The points (in the dataset) falling on the bounding planes are called ***support vectors*** because they are supporting the hyperplane. In fact we need only these points for finding the optimal hyperplane.

Situation 2: The training data has some outlier data due to which sometime there wont be a linear classifier which separates the data without training errors. See figure 8.1.5 to understand misclassification. You can see that there are few training data which has left the proximity of its class and gone towards the other class. Hence, misclassification occurs either due some training error (improper choice of classifier) or because our features are not that robust. In either case, we prefer that the training algorithm may simply minimize the number of misclassifications whilst maximizing the margin with respect to the correctly classified examples. We then say that, the SVM training algorithm allow training error.

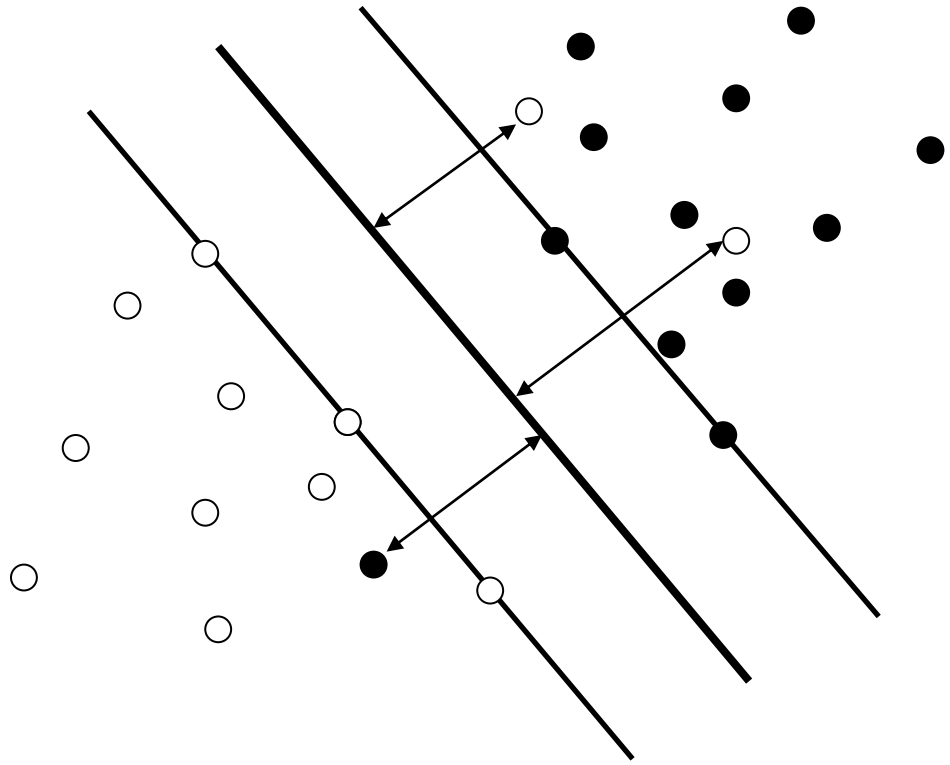


Fig. 8.1.5 Linear classifier with training error (Printed with permission from [5])

Situation 3: This is a real world situation and all the examples that we have solved in this book belong to this group. In this case the training data is clustered in such a fashion that the two classes are not at all linearly separable. See figure 8.1.6. That is, if we try for a linear classifier, it may have to tolerate large 'training error'.

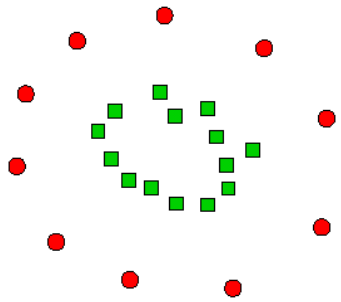


Fig. 8.1.6 Non linear spread of data

But if you bring non-linearity into the picture, you can tell quickly that a circular classifier would be perfect. It is easy for the human mind to tell that but for a computer it requires more

computational effort. This is where the Kernel Methods comes for the rescue. Kernel Methods approach the problem by mapping the data into some higher dimensional space called 'feature space', F, where it is linearly separable. See figure 8.1.7 where we have mapped the two dimensional data into three dimensional space in which the training data is clearly separable using a linear surface. To distinguish between these two spaces, we called the original space of data points as '**input space**' and the higher dimensional space as '**Feature space**'.

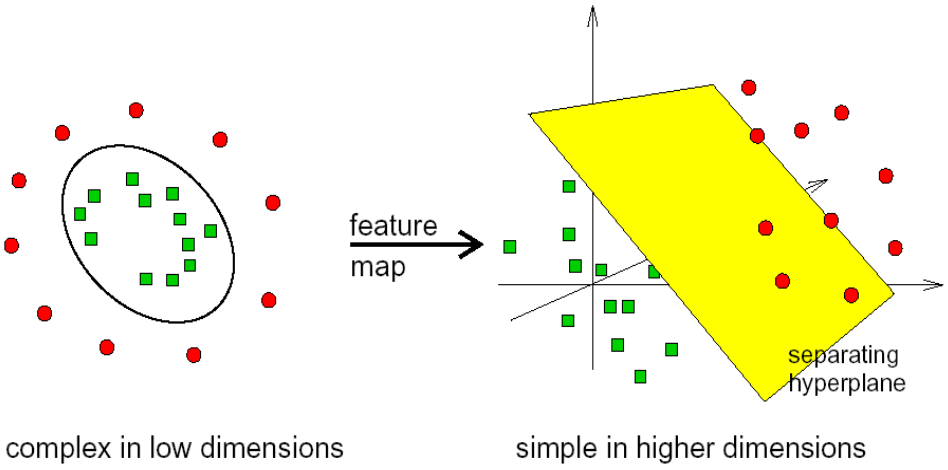


Fig. 8.1.7 Non linear classifier (Printed with permission from [5])

The hyper plane (n-dimensional plane) in 'Feature Space' corresponds to a highly nonlinear separating-surface in the original input space. Hence we call the classifier as non-linear classifier.

Note that, from a two dimension input point (x_1, x_2), we made three dimensional point (y_1, y_2, y_3) using a kernel function. For the above e.g., we used the kernel function as shown in figure 8.1.8. At this juncture, the reader may feel that, the process involves heavy computation especially when we map the data which itself may be of high dimensional, map to still higher (possibly infinite) dimensional feature space. However, we will see later that, we need not do any explicit mapping to higher dimensional space for finding the hyper plane classifier; all computation will be done in the input space itself. This is called as **Kernel trick**.

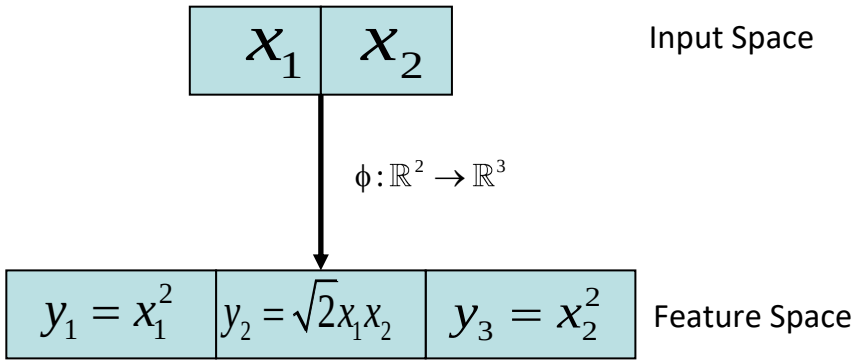


Fig. 8.1.8 Input space and Feature Space

There are several SVM algorithms in the literature and several new algorithms are constantly appearing year by year. In this book we are limiting ourselves to the formulation of SVM problems which we have used to solve the example problems in the coming chapters. Also the reader is advised to refer some optimization books to understand the numerical aspects of quadratic programming implementations and formulating dual optimization problem. We will use **Microsoft Excel** and show that how easily we can learn and develop the applications of machine learning.

8.1.4 SVM formulation with two variables

Consider 8 data points given in the following Table 8.1.2. Its plot is given in figure 8.1.9 (a). In the table, 'i 'is the index of the point and two variable values are given in 2nd and 3rd column. Fourth column gives the class value. Our aim is to find separating hyper plane of the form $w_1x_1 + w_2x_2 - \gamma = 0$ and two bounding planes of the form $w_1x_1 + w_2x_2 - \gamma \geq +1$ and. $w_1x_1 + w_2x_2 - \gamma \leq -1$. In SVM literature, w is often called as weight vector and γ is called the bias.

i	x_{i1}	x_{i2}	d_i
1	1	1	–1
2	2	1	–1
3	1	2	–1
4	2	2	–1
5	4	4	+1
6	4	5	+1
7	5	4	+1
8	5	5	+1

Table 8.1.2 Data for classification

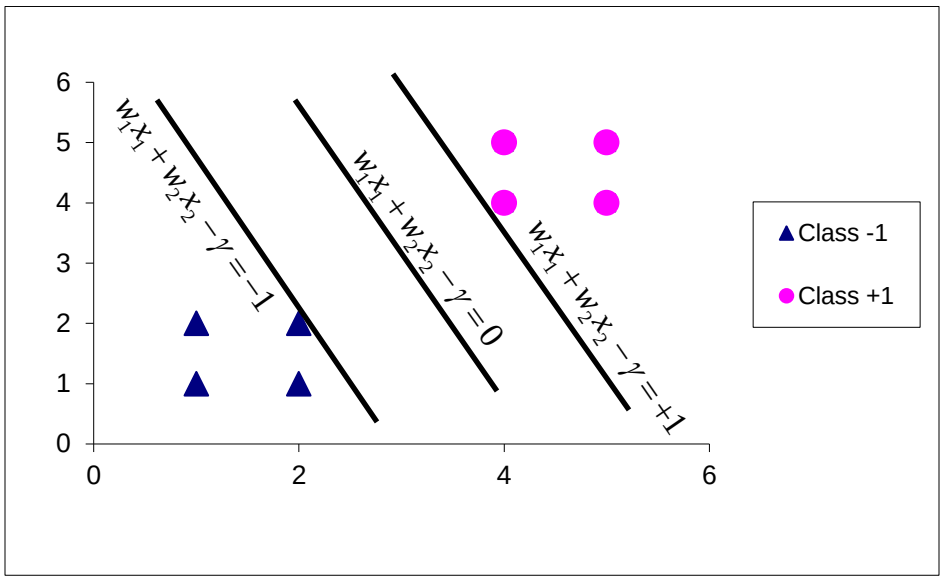


Fig. 8.1.9(a) Plot of data points

Note that these lines are parallel and even if we choose a more general bounding plane of the form $w_1x_1 + w_2x_2 - \gamma \geq +C$ and, $w_1x_1 + w_2x_2 - \gamma \leq -C$, they can be reduced to the form with +1 and –1 on right hand side. Our aim is to find maximally separating bounding planes such that data points with d = -1 satisfies the constraints

$$w_1x_1 + w_2x_2 - \gamma \leq -1$$

And data points with d = +1 satisfy

$$w_1 x_1 + w_2 x_2 - \gamma \geq +1$$

We write these constraints explicitly for each data points as:-

$1w_1 + 1w_2 - \gamma \leq -1$; or $(-1)\left(1w_1 + 1w_2 - \gamma\right) \geq +1$. This corresponding to first data point

$2w_1 + 1w_2 - \gamma \leq -1$ or $(-1)\left(2w_1 + 1w_2 - \gamma\right) \geq +1$

$1w_1 + 2w_2 - \gamma \leq -1$ or $(-1)\left(1w_1 + 2w_2 - \gamma\right) \geq +1$

$2w_1 + 2w_2 - \gamma \leq -1$ or $(-1)\left(2w_1 + 2w_2 - \gamma\right) \geq +1$

$4w_1 + 4w_2 - \gamma \geq +1$ or $(+1)\left(4w_1 + 4w_2 - \gamma\right) \geq +1$

$4w_1 + 5w_2 - \gamma \geq +1$

$5w_1 + 4w_2 - \gamma \geq +1$

$5w_1 + 5w_2 - \gamma \geq +1$

The resulting equation in matrix form is:

$$\begin{bmatrix} -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \left[\begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 1 & 2 \\ 2 & 2 \\ 4 & 4 \\ 4 & 5 \\ 5 & 4 \\ 5 & 5 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} - \gamma \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \right] \geq \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\boldsymbol{D}[\boldsymbol{A}\boldsymbol{w} - \gamma \boldsymbol{e}] \geq \boldsymbol{e}$$

where $\boldsymbol{e} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$, $\boldsymbol{A} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 1 & 2 \\ 2 & 2 \\ 4 & 4 \\ 4 & 5 \\ 5 & 4 \\ 5 & 5 \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_1^T \\ \boldsymbol{x}_2^T \\ \boldsymbol{x}_3^T \\ \boldsymbol{x}_4^T \\ \boldsymbol{x}_5^T \\ \boldsymbol{x}_6^T \\ \boldsymbol{x}_7^T \\ \boldsymbol{x}_8^T \end{bmatrix}$, $\boldsymbol{D} = \begin{bmatrix} -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$

The perpendicular distance of the bounding hyper plane $w_1 x_1 + w_2 x_2 - \gamma = +1$ from the origin is $\frac{|-\gamma - 1|}{\sqrt{w_1^2 + w_2^2}}$ and the distance to $w_1 x_1 + w_2 x_2 - \gamma = -1$ is $\frac{|-\gamma + 1|}{\sqrt{w_1^2 + w_2^2}}$. Therefore the total distance between the bounding hyper planes is $\frac{2}{\sqrt{w_1^2 + w_2^2}}$. Our aim is to find such a $\boldsymbol{w}$ and γ that maximizes this distance at the same time it should satisfy the matrix constraint equation. Maximization of $\frac{2}{\sqrt{w_1^2 + w_2^2}}$ is same as minimization of $\frac{w_1^2 + w_2^2}{2} = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w}$

So the learning problem is

$$\min_{\boldsymbol{w},\gamma} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w}$$
$$\text{Subject to } \boldsymbol{D}[\boldsymbol{A}\boldsymbol{w} - \gamma\boldsymbol{e}] \geq \boldsymbol{e} \qquad (8.1.1)$$

This formulation is true for any higher dimensional data vectors. The so called 'training of SVM' consists of finding $\boldsymbol{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$ and γ, given the matrix of data points ***A*** and the corresponding class vector **d.**

Once we obtain $\boldsymbol{w}$ and γ, we have our decision boundary as $\boldsymbol{w}^T\boldsymbol{x} - \gamma = 0$ or $w_1 x_1 + w_2 x_2 - \gamma = 0$ Since it is a two class problem and the test data point can either lie left side of the decision function or to the right side, hence the decision function is given by $f(\boldsymbol{x}) = \text{sign}(\boldsymbol{w}^T\boldsymbol{x} - \gamma)$. The formulation shown in Eq. 8.1.1 is called as the **Primal** formulation.

There is an alternate form to every primal formulation and that is called as dual formulation. Dual formulation is expressed in terms of **lagrangian multipliers**. The concept of dual formulation is out of scope of this book. It is advised for readers to refer any good texts on optimization to understand it fully. Expressing the problem in terms of lagrangian multipliers (dual variables) and solving it we get:-

$w = A^T Du$, where $u's$ are called the lagrangian multipliers. For each input vector x_i, there will be dual variable u_i. Hence w is expressed as the linear combination of u_i, x_i and d_i.

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = u_1 d_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + u_2 d_2 \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \ldots$$

In a typical problem most of $u_i's$ become zero and the data points corresponding to nonzero $u_i's$ are called support vectors**.** Support vectors always lie on the bounding planes. See figure 8.1.9 (b) for support vectors for the current problem.

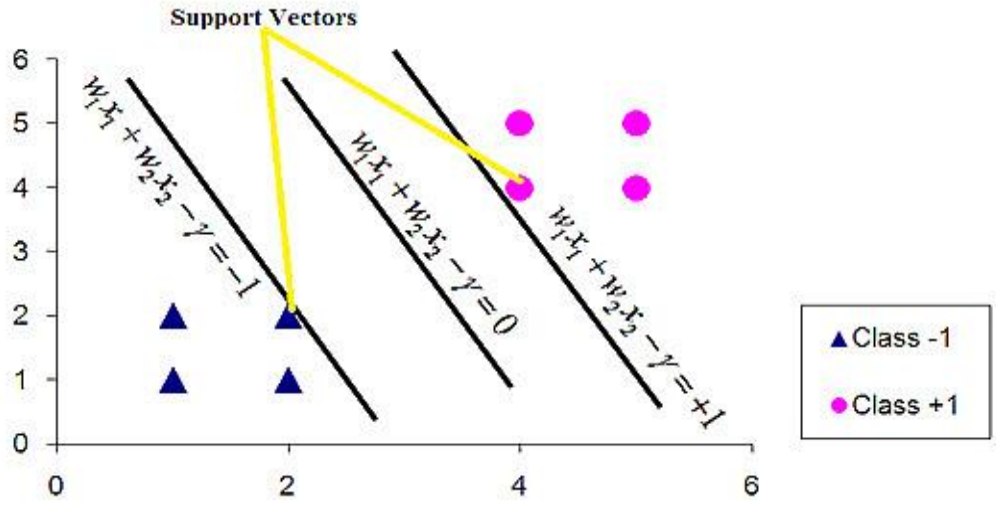


Fig. 8.1.9(b) Support Vectors

Substituting w in terms u, the final dual formulation is given as:-

$$Min\ L_D(u) = \frac{1}{2}u^T Qu - e^T u$$
$$s.t.\ d^T u = 0 \quad , u \geq 0$$
$$where\ \ Q = DAA^T D = (A * A^T).*(d * d^T) \qquad (8.1.2)$$

Note that D is the Diagonal matrix of class values, A is the data matrix and .* represents element by element multiplication.

The above optimization problem requires $D = \begin{bmatrix} d_1 & 0 & 0 & . \\ 0 & d_2 & 0 & . \\ 0 & 0 & d_3 & . \\ . & . & . & . \end{bmatrix}$ and

$AA^T = \begin{bmatrix} x_1^T x_1 & x_1^T x_2 & . & . \\ x_2^T x_1 & x_2^T x_2 & x_2^T x_3 & . \\ . & x_3^T x_2 & x_3^T x_3 & . \\ . & . & . & . \end{bmatrix}$ which is also called as **Kernel** since it contains all the required information. Mathematically the Kernel matrix has got a very important property. It is a symmetric positive semi definite matrix because of which the optimization problem always has a unique solution in terms of u.

Eq.8.1.2 yields $u_i s$ and once we obtain that, the decision function is given by

$$f(\boldsymbol{x}) = \text{sign}(\mathbf{w}^{\mathrm{T}}\mathbf{x} - \gamma) = \text{sign}(\sum_{\mathrm{i=1}}^{\mathrm{m}} u_i d_i \mathbf{x}_i^{\mathrm{T}}\mathbf{x} - \gamma)$$

$$\text{Or, } \mathrm{f(x)} = \text{sign}\left[\mathbf{(A^{T}Du)^{T}x - \gamma)} \right] = \text{sign}\mathbf{(u^{T}DAx} - \gamma)$$

γ can be easily calculated by taking a point on the margin for which $u_i \neq 0$ i.e. once you have identified a support vector on bounding plane suppose $w_1 x_1 + w_2 x_2 - \gamma = -1$, you can substitute the value of 'w' and find out γ .

8.1.5 Linear Kernel SVM with soft margin (Non Separable data)

Consider the problem depicted in figure 8.1.10. Here you can see that we can't find a linear separating boundary between the two classes with no error. So the only possibility way is to find a maximally separating plane with a small number of points that fall on the wrong side. The deviation of such a point from the respective bounding plane is called error. While training, we must try for maximum margin between the bounding planes and minimum number of points that contributes error. More the margin, more points will cause error. So we introduce a control parameter **C** through which we control the weight given to these two contradicting requirements. A **soft margin** problem means that we allow some amount error in the training process. While a hard margin problem does allow any error in the training.

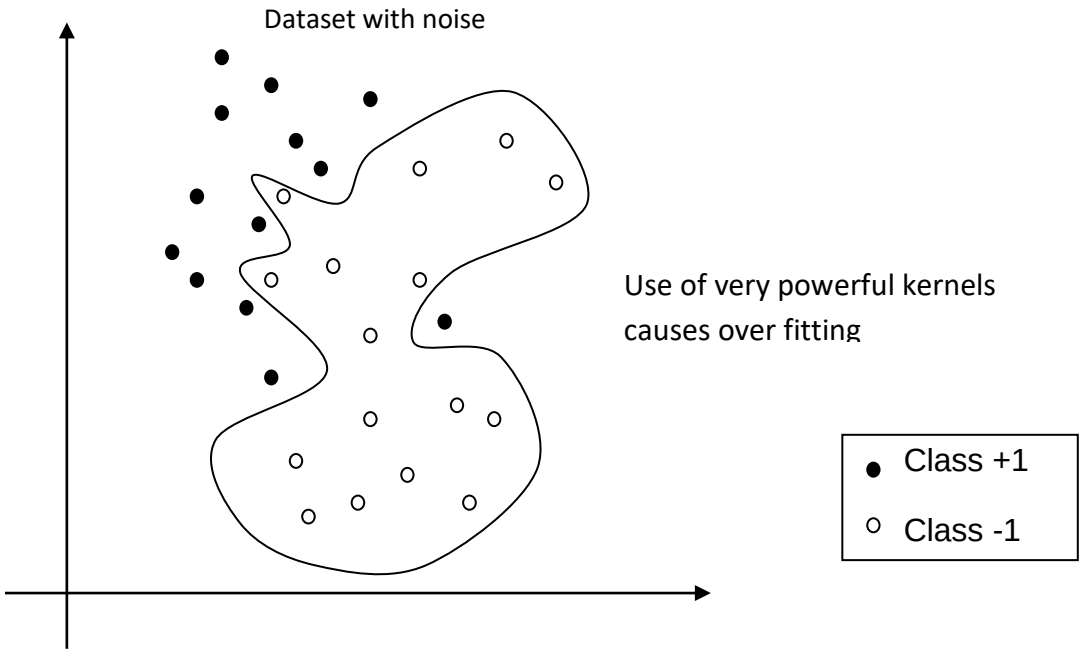


Fig. 8.1.10 Data which require linear SVM formulation with soft margin (boundary) (Printed with permission from [5])

Primal Formulation

$$\min_{\mathbf{w},\boldsymbol{\xi},\mathrm{C}} \; C\mathbf{e}^{\mathrm{T}}\boldsymbol{\xi} + \frac{1}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w}\text{, subject to the conditions} \qquad (8.1.3)$$

$$\mathbf{s.t}\ \mathbf{D}(\mathbf{A}\mathbf{w} - \mathbf{e}\gamma) + \boldsymbol{\xi} \geq \mathbf{e},$$

$$\boldsymbol{\xi} \geq \mathbf{0}$$

You see that the only changes made from linear formulation is by introducing parameter C and ξ. We add positive quantity ξ to the left of the inequality so that all points satisfies the constraint. Once an optimal classifier is found out, we can see that ξ corresponding to most points are zero. Only few points usually outliers require non zero ξ to satisfy the constraints. We can find its dual formulation using lagrangian method (Reader may refer the book [5]).

Dual Formulation

$$\text{Minimize } \mathrm{L}_{\mathrm{D}}(\mathbf{u}) = \frac{1}{2}\mathbf{u}^{\mathrm{T}}\boldsymbol{Q}\mathbf{u} - \mathbf{e}^{\mathrm{T}}\mathbf{u}$$

$$\mathbf{d}^{\mathrm{T}}\mathbf{u} = 0,\ \mathbf{0} \leq \boldsymbol{u} \leq C\boldsymbol{e} \qquad (8.1.4)$$

The main difference between soft margin and hard margin SVM Classifier (Eq. 8.1.2) is that in case of soft margin the lagrangian multiplier u_i s are bounded. That is

$$0 \leq u_i \leq C \quad ;\ i = 1,2,\ldots,m$$

Note: Parameter 'C' should be taken with due consideration as it impacts on the accuracy of the application.

8.1.6 Non Linear SVM and Kernel Trick

As we have already seen that if our data is not linearly is separable we may use a function $\phi(.)$ to map each data points x_i in to a higher dimensional space and then try for a maximally

separating linear hyperplane in that space as a classifier. The formulation is constructed in a similar way except that $\boldsymbol{x}_i$ gets modified into $\phi(x)$. The formulation looks like:-

Primal Formulation

$$\min_{\mathbf{w},\gamma,\boldsymbol{\xi}} \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\xi_i$$
$$\text{subject to } d_i(\mathbf{w}^T\phi(\mathbf{x}_i) - \gamma) + \xi_i - 1 \geq 0, \quad 1 \leq i \leq m$$
$$\xi_i \geq 0, \quad 1 \leq i \leq m$$

The corresponding dual formulation also changes in a similar fashion.

Dual Formulation

$$\text{Minimize } \mathrm{L_D}(\mathbf{u}) = \frac{1}{2}\mathbf{u}^{\mathrm{T}}\boldsymbol{Q}\mathbf{u} - \mathbf{e}^{\mathrm{T}}\mathbf{u}$$

$$\text{Subject to } \mathbf{d}^{\mathrm{T}}\mathbf{u} = 0\ ,\ \mathbf{0} \leq \boldsymbol{u} \leq C\boldsymbol{e} \qquad (8.1.5)$$

Here $\boldsymbol{Q} = \boldsymbol{DKD}$ and $\boldsymbol{K}$ is kernel matrix.

The decision function is given by $\mathbf{w}^T\phi(\mathbf{x}_i) - \gamma = 0$

Note: The dual formulation shown here is used in most of the examples shown in Chapter 9.

Now SVM algorithm for training requires $K = \phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j)$ for all i and j. If we are doing such explicit mapping for all input data vectors and doing dot product in the mapped higher dimensional space then computational requirement is huge especially when either dimension of data is high or number of data points is high or both are high. **Kernel trick** is about obtaining such dot products without explicitly mapping the data points into higher dimensional space. Thus the advantage SVM based non-linear classifiers is that:-

1) the training algorithm is same as that of linear classifier

2) and the problem of 'curse of dimensionality' is tackled in a simple way

Let's understand the Kernel Trick with a simple example

Consider solving the simplest 1-D classification problem (figure 8.1.11) given the input and the output (desired) values as follows: $\boldsymbol{A} = \begin{bmatrix} -1 = x_1 \\ 0 = x_2 \\ 1 = x_3 \end{bmatrix}, \boldsymbol{d} = \begin{bmatrix} -1 \\ 1 \\ -1 \end{bmatrix}$. Here first and third data point belongs to same class. Since it is not linearly separable, we go for non-linear classifier.

Here we choose the following mapping to the feature space: $\boldsymbol{\phi}(x) = \begin{bmatrix} \phi_1(x) \\ \phi_2(x) \\ \phi_3(x) \end{bmatrix} = \begin{bmatrix} x^2 \\ \sqrt{2}x \\ 1 \end{bmatrix}$ which will map a 1-D dimensional point into a 3-D space where it will be easy for find a linear separating surface between the two classes.

The mapping produces the following three points in the feature space (shown as the *rows* of the matrix **F** (*F* standing for features))

$$F = \begin{bmatrix} \boldsymbol{\phi}(x_1)^T \\ \boldsymbol{\phi}(x_2)^T \\ \boldsymbol{\phi}(x_3)^T \end{bmatrix} = \begin{bmatrix} 1 & -\sqrt{2} & 1 \\ 0 & 0 & 1 \\ 1 & \sqrt{2} & 1 \end{bmatrix}$$

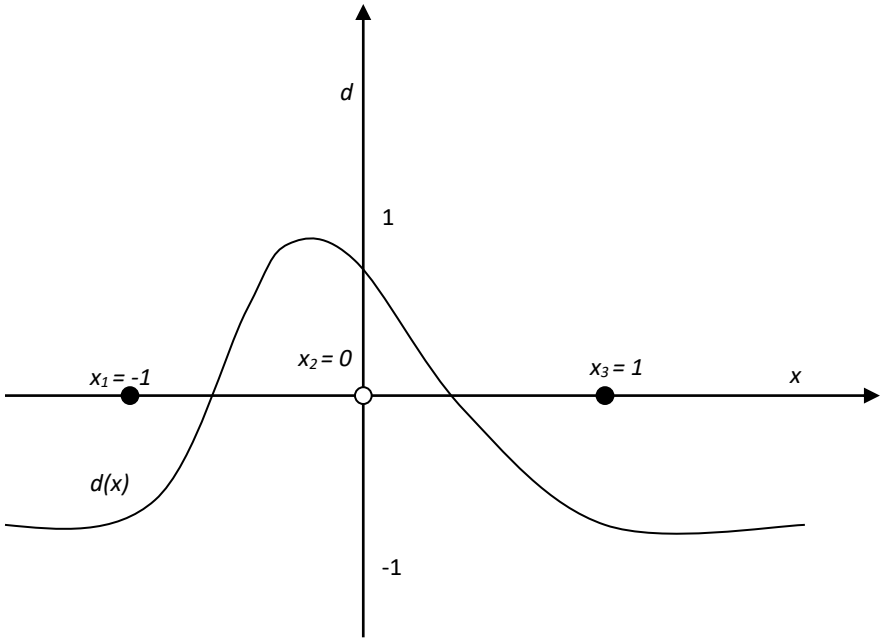


Fig. 8.1.11 A nonlinear 1 – dimensional classification problem. One possible solution is given by the decision function *d(x)* (solid curve) (Printed with permission from [5])

These three points are linearly separable in a feature space. It will become obvious if we note that, we have 3 axes viz x^2, x^1 and x^0 axes and values of points in the third axis x^0 is a constant 1. So if we plot using only first two axes, the points look like the one shown in figure 8.1.12. In the original domain the surface look like the one given in figure 8.1.11

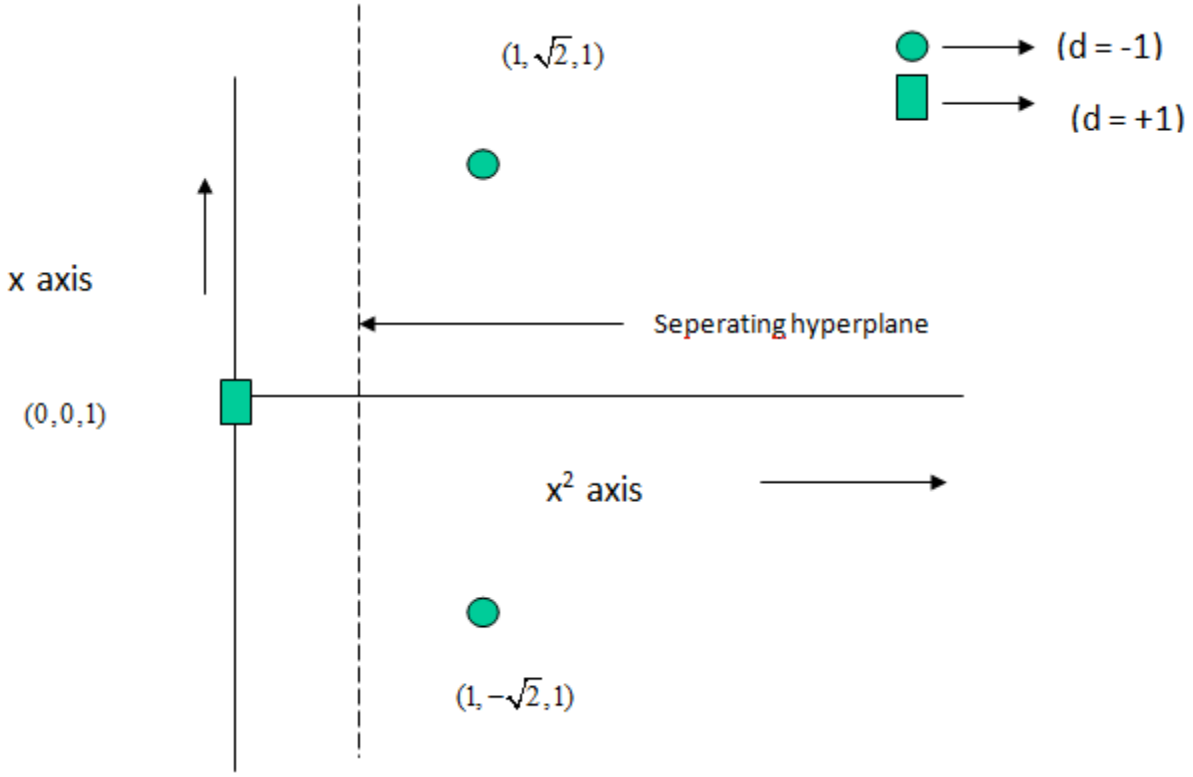


Fig. 8.1.12 Hyper surface in the feature space (Printed with permission from [5])

The Kernel ***K*** is given by

$$\boldsymbol{K} = \boldsymbol{F} * \boldsymbol{F}^{\mathrm{T}} = \begin{bmatrix} 1 & -\sqrt{2} & 1 \\ 0 & 0 & 1 \\ 1 & \sqrt{2} & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ -\sqrt{2} & 0 & \sqrt{2} \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 4 \end{bmatrix}$$

We now show that the mapping obtained by $\phi(x) = \begin{bmatrix} \mathrm{x}^2 \\ \sqrt{2}\mathrm{x} \\ 1 \end{bmatrix}$ is a scalar product implementation of

a quadratic kernel function $\left(x^T x + 1\right)^2 = \mathrm{k}(x_i, x_j)$.

$$\mathrm{K} = \begin{bmatrix} \phi(x_1)^T \phi(x_1) & \phi(x_1)^T \phi(x_2) & \phi(x_1)^T \phi(x_3) \\ \phi(x_2)^T \phi(x_1) & \phi(x_2)^T \phi(x_2) & \phi(x_2)^T \phi(x_3) \\ \phi(x_3)^T \phi(x_1) & \phi(x_3)^T \phi(x_2) & \phi(x_3)^T \phi(x_3) \end{bmatrix}$$

$$= \begin{bmatrix} (x_1^T x_1 + 1)^2 & (x_1^T x_2 + 1)^2 & (x_1^T x_3 + 1)^2 \\ (x_2^T x_1 + 1)^2 & (x_2^T x_2 + 1)^2 & (x_2^T x_3 + 1)^2 \\ (x_3^T x_1 + 1)^2 & (x_3^T x_2 + 1)^2 & (x_3^T x_3 + 1)^2 \end{bmatrix}$$

$$= \begin{bmatrix} (1+1)^2 & (0+1)^2 & (-1+1)^2 \\ (0+1)^2 & (0+1)^2 & (0+1)^2 \\ (-1+1)^2 & (0+1)^2 & (1+1)^2 \end{bmatrix} = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 4 \end{bmatrix}$$

Thus the Kernel ***K*** is be obtained first by taking a dot product in the original domain , then adding a 1 to the product and then squaring the resulting sum.

Note that a *kernel function* $K(x_i, x_j)$ is a function in input space. Thus, the basic advantage in using kernel function $K(x_i, x_j)$ is in avoiding performing a mapping $\phi(x)$ at all. In this way, we bypass a possibly extremely high dimensionality of a feature space *F* and this is called as **Kernel Trick**.

8.1.7 Some Common Types of Kernels

a. Polynomial kernels

A polynomial Kernel of degree '*p*' is computed as

$$K = \begin{bmatrix} (x_1^T x_1 + 1)^p & (x_1^T x_2 + 1)^p & . & (x_1^T x_m + 1)^p \\ . & . & (x_i^T x_j + 1)^p & . \\ . & . & . & . \\ (x_m^T x_1 + 1)^p & (x_m^T x_2 + 1)^p & . & (x_m^T x_m + 1)^p \end{bmatrix}$$

Note that dot product is taken in the original domain.

b. Radial basis function Kernels

The radial basis function is given by $k(\boldsymbol{x}, \boldsymbol{y}) = \exp(-\sigma \|\boldsymbol{x} - \boldsymbol{y}\|^2)$ where σ is a positive parameter controlling the radius. Expanding $\exp(-\sigma \|\boldsymbol{x} - \boldsymbol{y}\|^2)$ we obtain

$$\exp(-\sigma \|\boldsymbol{x} - \boldsymbol{y}\|^2) = \exp(-\sigma \|\boldsymbol{x}\|^2) \exp(-\sigma \|\boldsymbol{y}\|^2) \exp(2\sigma \boldsymbol{x}^T \boldsymbol{y}).$$

Since $\exp(2\sigma \boldsymbol{x}^T \boldsymbol{y}) = 1 + 2\sigma \boldsymbol{x}^T \boldsymbol{y} + \frac{(2\sigma)^2}{2!} (\boldsymbol{x}^T \boldsymbol{y})^2 + \frac{(2\sigma)^3}{3!} (\boldsymbol{x}^T \boldsymbol{y})^3 + \ldots\ldots,$

$\exp(2\sigma \boldsymbol{x}^T \boldsymbol{y})$ is an infinite summation of polynomials . Thus it is a kernel whose mapping function maps a point to an infinite dimensional space.

Note: RBF is the most commonly used kernel.

8.1.8 Normalization of Kernels

In many practical applications like text categorization, the number of variable is very large and the value of kernel may become too small or big and training of support vector machines become too difficult. To overcome this, it is advisable to normalize each variable and also the kernel.

Variable can be normalized to [0,1] range or [-3 , +3] using the formula $(x_i - \mu_i)/\sigma_i$, where μ_i and σ_i are the mean and standard deviation of the i[th] predictor variable.

8.2 Variants of SVM - Mangasarian's Approaches

The SVM formulation that we have seen until now is quadratic in nature and hence it requires a Quadratic Solver. Professor Olvi Mangasarian [8] made modification on the classical SVM which requires only linear matrix solver for the solution. This reduces a great deal of computational effort. In terms of accuracy, its performance is comparable with classical SVM.

Mangasarian [1] changed the classical formulation SVM

$$\min_{\mathbf{w},\gamma,\boldsymbol{\xi}} \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\xi_i$$
$$\text{subject to } d_i(\mathbf{w}^T\mathbf{x}_i - \gamma) + \xi_i - 1 \geq 0, \quad 1 \leq i \leq m$$
$$\xi_i \geq 0, \quad 1 \leq i \leq m$$

into

$$\min_{\mathbf{w},\gamma,\boldsymbol{\xi}} \sum_{i=1}^{m}\xi_i$$
$$\text{subject to } d_i(\mathbf{w}^T\mathbf{x}_i - \gamma) + \xi_i - 1 \geq 0, \quad 1 \leq i \leq m$$
$$\xi_i \geq 0, \quad 1 \leq i \leq m$$

He completely removed the quadratic term from the objective function and showed that this formulation gave comparable results. Since everything is linear now, we can use linear solver to get the solution.

In matrix form this can be written as

$$\min_{\boldsymbol{w},\boldsymbol{\xi},\gamma} \boldsymbol{e}^T\boldsymbol{\xi}$$
$$\boldsymbol{D}(\boldsymbol{A}\boldsymbol{w} - \boldsymbol{e}\gamma) + \boldsymbol{\xi} \geq \boldsymbol{e}$$
$$\boldsymbol{\xi} \geq \boldsymbol{0}$$

We have seen that using lagrangian method, we obtain $\boldsymbol{w} = \boldsymbol{A}^T\boldsymbol{D}\boldsymbol{u}$. Substituting it we get:-

$$\min_{\boldsymbol{u},\boldsymbol{\xi},\gamma} \boldsymbol{e}^T\boldsymbol{\xi}$$
$$\boldsymbol{D}(\boldsymbol{A}\boldsymbol{A}^T\boldsymbol{D}\boldsymbol{u} - \boldsymbol{e}\gamma) + \boldsymbol{\xi} \geq \boldsymbol{e}$$
$$\boldsymbol{\xi} \geq \boldsymbol{0}$$

Now we replace $\boldsymbol{A}\boldsymbol{A}^T$ by $\boldsymbol{K}$ to obtain a non-linear kernel version of SVM

$$\min_{\boldsymbol{u},\boldsymbol{\xi},\gamma} \boldsymbol{e}^T\boldsymbol{\xi}$$
$$\boldsymbol{D}(\boldsymbol{K}\boldsymbol{D}\boldsymbol{u} - \boldsymbol{e}\gamma) + \boldsymbol{\xi} \geq \boldsymbol{e}$$
$$\boldsymbol{\xi} \geq \boldsymbol{0}$$

The formulation can be further simplified to

$$\min_{\boldsymbol{u},\boldsymbol{\xi},\gamma} \boldsymbol{e}^T\boldsymbol{\xi}$$
$$\boldsymbol{Qu} - \boldsymbol{d}\gamma + \boldsymbol{\xi} \geq \boldsymbol{e}, \text{ where } \boldsymbol{Q} = \boldsymbol{K}.*(\boldsymbol{d} * \boldsymbol{d}^T)$$
$$\boldsymbol{\xi} \geq \boldsymbol{0}$$

This is the linear programming version of standard SVM. Once we obtain $\boldsymbol{u}$ and γ, the decision function is given by

$$f(x) = sign\left(\sum_{i \in svmindex} d_i u_i \mathrm{K}(x_i, x) - \gamma \right)$$

8.3 Support Vector Regression (SVR)

In the previous section we saw how the classification problems are handled by the SVM. Here we see how the regression problems are handled where we need to learn a function from set training vectors which approximates the pattern of input dataset. Regression belongs to unsupervised learning where there is no output class information. SVR retains all the properties that characterize maximal margin algorithms of support vector machines.

Regression [5] is used in biological contexts to model disease onset and intensity as influenced by various behaviors and environmental conditions. It has also been used for areas such as time series analysis, drug discovery, civil engineering, and sunspot frequency prediction. Image tracking, Image Compression, Control systems and System Identification Problems are areas where newer applications of Support vector regression are being pursued.

Before we get into the formulation aspect, we need to understand the concept of loss functions.

8.3.1 Loss Function

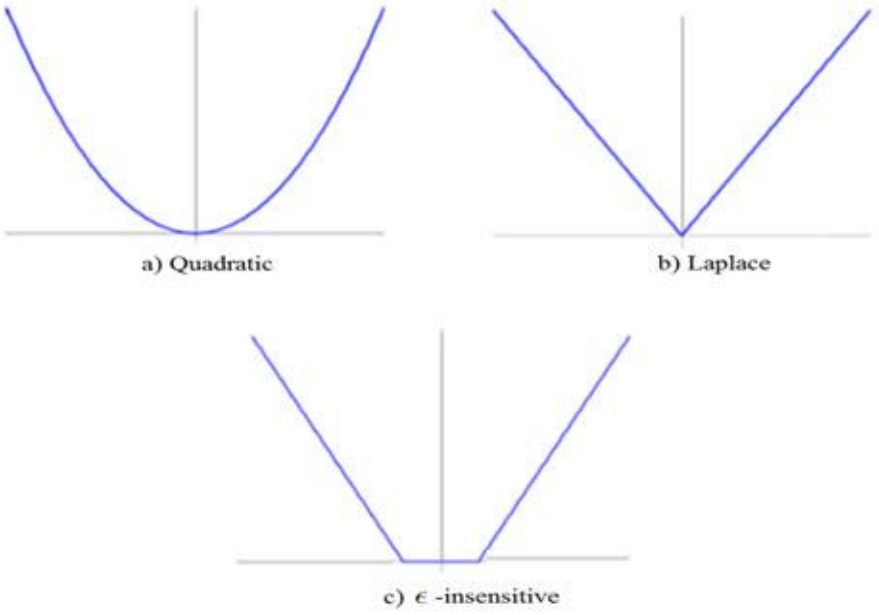


Fig. 8.3.1 Different Loss Functions

Loss function is a measure of amount of penalty being applied to a system when it encounters an error. As always, when the error increases, the penalty increases. But there are different loss functions as shown in figure 8.3.1. The loss function in figure 8.3.1(a) corresponds to the conventional least squares error criterion. The loss function in figure 8.3.1(b) is a Laplacian loss function that is less sensitive to outliers than the quadratic loss function. The loss function given in figure 8.3.1(c) is called the ε**-insensitive** loss function**.** It is proposed by Vapnik and is more robust for the outliers and also when the underlying distribution of data is unknown [9].

The concept of loss function is very important in the context of SVR. Different loss functions result in different overall strategy in the learning. We talked about generalization error in SVM, the same is applicable here. That means sometimes the small residual errors may be inevitable to decrease the generalization error.

Support vector regression [5] performs linear regression in the feature space using ε-insensitive Loss function given by:-

$$L_{\varepsilon}(y, f(\boldsymbol{x}, \boldsymbol{w})) = \begin{cases} 0 & if\ |y - f(\boldsymbol{x}, \boldsymbol{w})| \leq \varepsilon \\ |y - f(\boldsymbol{x}, \boldsymbol{w})| & otherwise \end{cases} \tag{8.3.1}$$

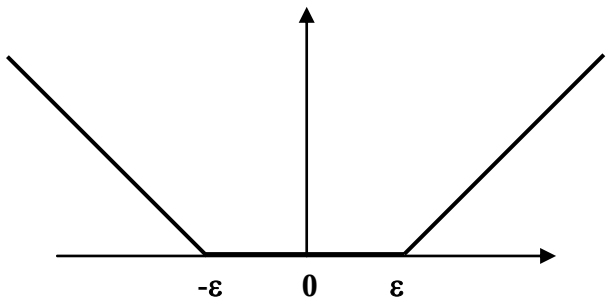


Fig. 8.3.2 ε insensitive loss function

Title on x axis: $\left(y - f(\boldsymbol{x}, \boldsymbol{w})\right)$

Title on y axis: $L_{\varepsilon}\left(y - f(\boldsymbol{x}, \boldsymbol{w})\right)$

Here ε is a constant which is fixed before training. Let us also introduce (non-negative) slack variables ξ_i, ξ_i^* $i = 1,...n$, to measure the deviation of training samples outside ε-insensitive zone. figure 8.3.3 below shows how the ε-insensitive zone look like when the regression is linear

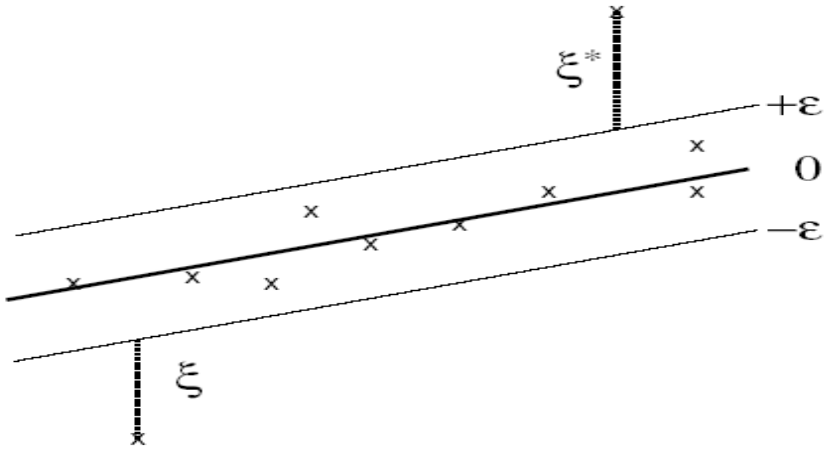


Fig. 8.3.3 ε-insensitive zone for linear support vector regression

If a point $\boldsymbol{x}_i$ falls within ε-insensitive zone, the associated ξ_i, ξ_i^* is zero. If it is above the ε-insensitive zone, $\xi_i^* > 0, \xi_i = 0$ and $\xi_i^* = 0, \xi_i > 0$ if the point is below it.

The function to be learned is $y = \boldsymbol{w}^T \boldsymbol{x} + \gamma$ for linear regression and $y = \boldsymbol{w}^T \phi(\boldsymbol{x}) + \gamma$ for non-linear case.

8.3.2 Linear Support Vector Regression Formulations

Thus SVM regression is formulated as minimization of the following functional:

$$\text{Min} \quad \frac{1}{2} \| \boldsymbol{w} \|^2 + C \sum_{i=1}^{n} (\xi_i + \xi_i^*)$$

Subject to

$$\begin{cases} \boldsymbol{w}^T \boldsymbol{x_i} - y_i \le \varepsilon + \xi_i \\ y_i - \boldsymbol{w}^T \boldsymbol{x_i} \le \varepsilon + \xi_i^* \\ \xi_i \, , \xi_i^* \ge 0, i = 1, \ldots, n \end{cases} \tag{8.3.2}$$

The presence of $\frac{1}{2} \| \boldsymbol{w} \|^2$ in the objective function makes it a quadratic optimization problem. Its role $\frac{1}{2} \| \boldsymbol{w} \|^2$ in the objective function is to make the target function (function to be learned from data for prediction) less complex thereby avoiding the problem of over fitting. y_i is the actual value while $\boldsymbol{w}^T \boldsymbol{x_i}$ is the predicted value. The residual error is defined by $\pm(\boldsymbol{w}^T \boldsymbol{x_i} - y_i)$. If the kernel is linear, it can be solved in the primal. Otherwise, we solve the problem in its dual variables. In either case, the problem remains a quadratic optimization problem (QP).

8.3.3 Non-Linear Support Vector Regression Formulations

As in the previous cases, SV regression is formulated as minimization of the following functional:

$$\text{Min} \quad \frac{1}{2} \| \boldsymbol{w} \|^2 + C \sum_{i=1}^{n} (\xi_i + \xi_i^*)$$

$$\text{s.t.} \quad \begin{cases} \boldsymbol{w}^T \phi(\boldsymbol{x_i}) - y_i \le \varepsilon + \xi_i \\ y_i - \boldsymbol{w}^T \phi(\boldsymbol{x_i}) \le \varepsilon + \xi_i^* \\ \xi_i \, , \xi_i^* \ge 0, i = 1, \ldots, m \end{cases} \tag{8.3.3}$$

$\phi(\boldsymbol{.})$ Maps the data point $\boldsymbol{x}$ into high-dimensional feature space and linear regression (with ε-insensitive loss function) is performed in that feature space. We will try to express the target function in terms of dual variables.

Using lagrangian dual, we get

$$\boldsymbol{w} = \sum_{i=1}^{m}\left(u_i^* - u_i\right)\phi(\boldsymbol{x}_i)$$

$\eta_i = C - u_i$

$\eta_i^* = C - u_i^*$, where $u_i^*, u_i, \eta_i, \eta_i^*$ are the dual variables. Substituting the primal variables $\boldsymbol{w}$ and dual variables η_i, η_i^* into (8.3.3) yields dual optimization problem

$$\text{Max } L = \frac{-1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\left(u_i^* - u_i\right)\left(u_j^* - u_j\right)k_{ij} - \varepsilon\sum_{i=1}^{m}\left(u_i^* + u_i\right) + \sum_{i=1}^{m} y_i\left(u_i^* - u_i\right)$$

Subject to $0 \le u_i, u_i^* \le C$

Here $k_{ij} = \phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j)$

Finally converting to minimization form yields

$$\text{Min } L = \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\left(u_i^* - u_i\right)\left(u_j^* - u_j\right)k_{ij} + \varepsilon\sum_{i=1}^{m}\left(u_i^* + u_i\right) - \sum_{i=1}^{m} y_i\left(u_i^* - u_i\right) \qquad (8.3.4)$$

Subject to $0 \le u_i, u_i^* \le C$

Matrix Formulation

Let $\bar{\boldsymbol{u}} = \begin{bmatrix} \boldsymbol{u}^* \\ \boldsymbol{u} \end{bmatrix}$, $\boldsymbol{g} = \begin{bmatrix} \varepsilon - \boldsymbol{y} \\ \varepsilon + \boldsymbol{y} \end{bmatrix}$

The formulation (8.3.4) can be written as

$$\underset{\bar{u}}{\text{Min}}\ \text{L} = \frac{1}{2}\bar{\boldsymbol{u}}^T\boldsymbol{Q}\bar{\boldsymbol{u}} + \boldsymbol{g}^T\bar{\boldsymbol{u}}$$

Subject to $0 \le u_i, u_i^* \le C$

Where $\boldsymbol{Q} = \begin{bmatrix} \boldsymbol{K} & \boldsymbol{-K} \\ \boldsymbol{-K} & \boldsymbol{K} \end{bmatrix}$

The regression function is given by $y = \boldsymbol{w}^T \phi(\boldsymbol{x}) + \gamma$ .Since $\boldsymbol{w} = \sum_{i=1}^{m} \left(u_i^* - u_i\right) \phi(\boldsymbol{x}_i)$, the regression function changes to, $y = f(x) = \sum_{i=1}^{m} \left(u_i^* - u_i\right) \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x})$.

8.4 Multi-class Support Vector Machines

Till now we studied all about binary classifiers where we have only two output classes. In this chapter we will see how to extend the concept to Multi-class problems. There are two types of approaches for multiclass SVM:-

8.4.1 Indirect methods

a) One against All Method

In one against all (OAA) method, for N classes, N classifiers are made. A data point would be classified under a certain class if and only if that class's SVM accepted it and all other classes' SVMs rejected it. For clearly separated classes, this method works fine else there will be a problem when more than one class accepts while all others reject it. This problem is mainly due to the bad selection of features. One method for improving the performance of OAA SVMs was suggested by Vapnik in 1998. It involves using the continuous values of SVM decision functions rather than simply their signs. The class of a data point is whichever class has a decision function with highest value, regardless of sign. This appears to be the most common method for multiclass SVM classification in use today.

b) One against One Method or Pair wise Methods

All the pairs present in N classes are considered. Hence for N classes N (N-1)/2 binary classifiers are built, one for each pair of classes. The prediction of class for a new data point is based on voting scheme.

8.4.2 Direct Methods

Examples of direct approaches were presented by Vapnik, Weston and Watkins [10] and Crammer and Singer [11]. Intuitively they attempt to find separating boundaries for all the classes in one step. For each class they define a decision rule similar to that of binary SVMs and during testing the test data point is assigned the label of the decision rule that yielded the highest (positive) margin. Such methods are however numerically difficult due to the large number of variables that needs to be optimized and are rather difficult to implement.

8.4.3 Formulation of the SVM with Vector Output

This section discusses the formulation as given in [12]. First of all, we will analyze how multi-class formulation and interpretation differ from classical binary SVMs.

Firstly, class labels are vectors instead of +1s and –1s in the binary SVM. Thus class labels in binary SVM belong to one-dimensional subspace where as in the case of multi-class SVM, the class label belongs to multi-dimensional sub-space.

Secondly, **W**, which defines the separating hyper-plane in binary SVM, is a vector. In multiclass, **W** is a matrix. It can be assumed that the job of ***W*** in two-class SVMs is to map the data/feature vector into one-dimensional sub-space. In multi-class SVM, the natural extension of this function is then the mapping of data/feature space into vector label space whose defining basis are vectors. In other words, multi-class learning may be viewed as vector-labeled learning or vector value- learning.

The Support Vector Machine with vector output is realized on this sample by the following optimization problem:

$$\min \frac{1}{2}\mathrm{tr}(\mathbf{W}^{\mathrm{T}}\mathbf{W}) + \frac{1}{2} C\xi^{T}\xi$$

Subject to $\{\mathbf{W}|\mathbf{W}: H_{\phi(\mathbf{x})} \rightarrow H_{y},\ \mathbf{W} \text{ linear operator}\}$, (a)

$\{\xi|\xi \in H_{\mathrm{m}}\}$,slack or error vector (b)

$\left\langle \mathbf{y}_{\mathrm{i}}, (\mathbf{W}\phi(\mathbf{x}_{\mathrm{i}}))\right\rangle = 1 - \xi_{\mathrm{i}},\ \mathrm{i} = 1, \ldots, \mathrm{m},$ (c) (8.4.1)

Equation (8.4.1) is basically a modification of the formulation given by [12] in terms of Fung. and Mangasarian (2001), [13] approach for a two-class SVM.

Through introduction of the dual variables $\{\alpha_i \mid i = 1, \ldots m\}$ to the margin constraints and on the basis of Karush-Kuhn-Tucker theory, the linear operator **W** can be expressed by using the tensor products of the output and the feature vectors, that is:

On taking the Lagrangian, we obtain $\mathbf{W} = \sum_{i=1}^{m} \alpha_i \mathbf{y}_i \phi(\mathbf{x})^T$ and $\xi = \frac{\alpha}{C}$ (8.4.2)

Substituting it in the constraint (c) in (Equation 8.4.1), we obtain:

$$(\mathbf{K}^y .* \mathbf{K}^\phi)\alpha = \mathbf{e} - \frac{\alpha}{C} \quad (.* \text{ represents element-wise multiplication})$$

$$\text{or , } \mathbf{K}\alpha = \mathbf{e} - \frac{\alpha}{C}$$

where $\mathbf{K} = \mathbf{K}^y .* \mathbf{K}^\phi$ (.* represents element-wise multiplication)

and $K^\phi{}_{ij} = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ and $K^y{}_{ij} = \langle (\mathbf{y}_i, \mathbf{y}_j) \rangle$, stands for the elements of the kernel matrices for the feature vectors and for the label vectors, respectively.

$$(\mathbf{K} + \frac{\mathbf{I}}{C})\alpha = \mathbf{e}$$

$$\mathbf{Q}\alpha = \mathbf{e} \quad \text{where} \quad \mathbf{Q} = (\mathbf{K} + \frac{\mathbf{I}}{C}) \tag{8.4.3}$$

$$\alpha = \mathbf{Q}^{-1}\mathbf{e}$$

Using α, one can find the matrix W which will help for prediction as shown in the next section.

8.4.4 Multi-class Classification Prediction

The multi-class classification can be implemented within the framework of the vector valued SVM. Let us assume that the label vectors are chosen out of a finite set $\{\hat{\mathrm{y}}_1, \ldots, \hat{\mathrm{y}}_\mathrm{T}\}$ in the learning task. The decision function predicting one of these labels can be expressed by using the predicted vector output [12].

$$d(\mathbf{x}) = \arg \max_{t=1,\ldots,T} \left\langle \hat{\mathbf{y}}_t, \mathbf{W}\phi(\mathbf{x}) \right\rangle \tag{8.4.4}$$

$$= \arg \max_{t=1,\ldots,T} \sum_{i=1}^{m} \alpha_i K^y(\hat{\mathbf{y}}_t, \mathbf{y}_i) K^\phi(\mathbf{x}_i, \mathbf{x})$$

Now we are able to set up a multi-class classification. In [12], the following two rules are given for setting up the labels:

$$(\mathbf{y}_i)_t = \begin{cases} 1 & \text{if item } i \text{ belongs to category } t,\ t = 1, \ldots, T, \\ 0 & \text{otherwise.} \end{cases} \tag{8.4.5}$$

$$(y_i)_t = \begin{cases} \sqrt{\dfrac{T-1}{T}} & \text{if item } i \text{ belongs to category } t,\ t = 1, \ldots, T, \\ -\dfrac{1}{\sqrt{T(T-1)}} & \text{otherwise,} \end{cases} \tag{8.4.6}$$

For example, for a three-class classification problem, according to Equation (8.4.5),

$$y_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix},\ y_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix},\ y_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

and according to Equation (8.4.6):

$$y_1 = \begin{bmatrix} \sqrt{2/3} \\ -1/\sqrt{6} \\ -1/\sqrt{6} \end{bmatrix},\ y_2 = \begin{bmatrix} -1/\sqrt{6} \\ \sqrt{2/3} \\ -1/\sqrt{6} \end{bmatrix},\ y_3 = \begin{bmatrix} -1/\sqrt{6} \\ -1/\sqrt{6} \\ \sqrt{2/3} \end{bmatrix}$$

It was found that both choices work equally well in all the experiments that we carried out with benchmarking data-sets.

8.5 Summary

In this chapter we have seen how SVM learns the pattern present in the input data for both supervised and unsupervised problems. The problems present in the non-linear datasets are handled in an efficient and intuitive manner using Kernel methods. We explained the basic formulation required for two class, multi class, regression and Mangasarian approaches.

Questions:

1. Say you are given the data set shown figure 1. This is a binary classification task in which the instances are described by two real-valued attributes.

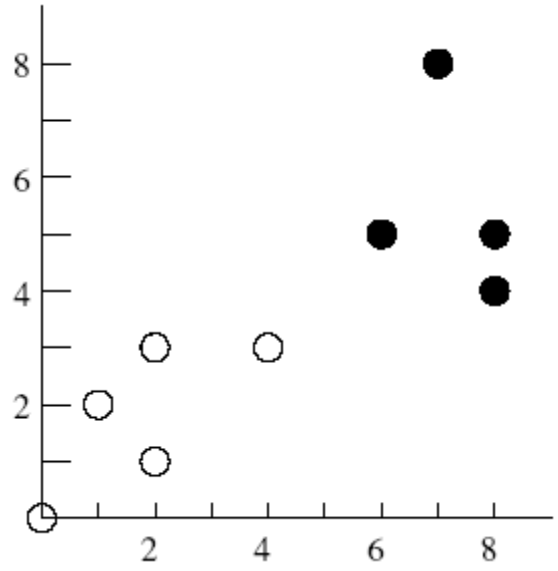


Fig 1 Input data (Printed with permission from [5])

(a) If the data in the graph are taken to be training data, which of the instances should be identified as support vectors?

(b) Use any one of the SVM algorithm to find lagrangian multipliers $u_i s$ (do not normalize the data).

c) draw the decision surface (a line here) and the two bounding planes

2. Generate a two-dimensional example (assume two real-valued attributes and two possible classes) that cannot be separated by a linear hyperplane, but that can be separated by a polynomial kernel with degree two.

(a) Give a graph representing your dataset.

(b) Give the training error with the linear kernel and with the polynomial kernel of degree two.

3. Develop a statistical model for forecasting the peak of the flood-wave of the Danube at Budapest. The following 24 data triplets registered from the year of 1896 up to 1955 are available. For details about the problem readers may refer [14]

The measured data are

X (rainfall in mm)	Y (water level at the beginning of raining in cm)	Z (peak of water level in cm)
58	405	590
52	450	660
133	350	780
179	285	770
98	330	710
72	400	640
72	550	670
43	480	520
62	450	660
67	610	690
64	380	500
33	460	460
57	425	610
62	560	710
54	420	620
48	620	660
86	390	620
74	350	590
95	570	740

44	710	730
77	580	720
46	700	640
123	560	805
62	4330	673

Table 1 Input data

The model $z = f(x, y)$ should generalize these data and ensure forecasting z from measured x and y values.

References

[1] History of Support Vector Machines", http://www.svms.org/history.html.

[2] B.E. Boser, I. Guyon, and V. Vapnik. "*A training algorithm for optimal margin classifiers"*. In Proc. of the Fifth Annual Workshop on Computational Learning Theory, pages 144--152, 1992.

[3] Nadaraya, E. "*On estimating regression".* Theory of Probab. and its Appl. 9,141–142.1964

[4] Watson, G. Smooth regression analysis. Sankhya, Series A 26, 302–305,1964

[5] K.P Soman., Loganathan.R., Ajay. V: *Machine Learning with SVM and Other Kernel Methods :* ISBN: 978-81-203-3435-9.

[6] V.N. Vapnik and A.Y. Chervonenkis. "*A note on one class of perceptrons*." Automation and Remote Control, 25, 1964.

[7]. V.N. Vapnik. "*Theory of Pattern Recognition",* Nauka, Moscow, 1974

[8] Olvi L. Mangasarian, http://pages.cs.wisc.edu/~olvi, Research Scientist Department of Mathematics, University of California, San Diego

[9] Steve R. Gunn, Technical Report, "*Support Vector Machines for Classification and Regression"*.
[10] J. Weston and C. Watkins. "*Multi-class support vector machines"*. Technical Report CSD-TR-98-04, Department of Computer Science, Royal Holloway, University of London, Egham, TW20 0EX,UK,1998.

[11] Koby Crammer and Yoram Singer. "*On the Algorithmic Implementation of Multiclass Kernel-based Vector Machines"*. Journal of Machine Learning Research 2 (2001), pp. 265-292

[12] S. Szedmak and J. Shawe-Taylor. "*Multiclass Learning at One-class Complexity"*. Technical Report, ISIS Group, Electronics and Computer Science , 2005.

[13] G. Fung and O.L. Mangasarian. "*Proximal Support Vector Machine Classifiers*". KDD 2001: Seventh ACM SIGKDD International Conference on Knowledge Discovery & Data Mining San Francisco August 26-29, 2001

[14] Bela Palanczi et al , "*Support Vector regression via Mathematica"* , Periodica Polytechnica Ser. Civ. Eng.. Vol. 49, No. 1, PP. 59–84 (2005)